

THE DOM, ON PAPER

Everything the cycles use and had no time to list, and every lecture slide the cycles replaced. This deck is not taught — it is looked up, at home, and in week 13 when there is no internet.

WHAT IS HERE

The catalogues: selecting, building, events, forms, DevTools.

WHAT IS NOT

Anything you are examined on that you did not type in a cycle. The spine points here with ↗ Reference; nothing here is new material for the assignment.

HOW TO USE IT

Print it, or keep the PDF open beside the editor. The last slide is the one to read first when something is broken.

THREE QUESTIONS, AND THE WHOLE DOM API ANSWERS THEM

WHERE IS IT?

`querySelector`, `closest`,
`traversal`. Cycles 1 and 3.

HOW DO I CHANGE IT?

`createElement`, `textContent`,
`append`, `classList`. Cycle 1.

HOW DO I HEAR THE USER?

`addEventListener`, the event
object, delegation. Cycles 2 and 3.

► Then cycle 5, the half you will still be using in ten years: **HOW TO FIND A BUG ON PURPOSE.**

THE FILE AND THE DOCUMENT ARE TWO DIFFERENT THINGS

left: what the server sent · right: the tree, walked right now ↻ reload open ↗

WHAT THE SERVER SENT · WHAT THE DOCUMENT IS NOW

Reading list

- Eloquent JavaScript

add a row set an attribute change the heading text insert a comment node reset

THE FILE, AS SERVED (FETCHED ONCE, NEVER RE-READ)

```
<div id="stage">
  <h3>Reading list</h3>
  <ul id="rows">
    <li>Eloquent JavaScript</li>
  </ul>
</div>
```

THE TREE, RIGHT NOW

```
<div id="stage"
  #text ·whitespace·
  <h3>
    #text "Reading list"
    #text ·whitespace·
    <ul id="rows"
      #text ·whitespace·
      <li>
        #text "Eloquent JavaScript"
        #text ·whitespace·
      #text ·whitespace·
    </li>
  </h3>
</div>
```

nodes	elements	text nodes	of them whitespace only
11	4	7	5

stage.firstChild is **#text (whitespace)** · stage.firstElementChild is **<h3>**. The indentation in the file is a text node, and it is why the first one surprises people.

► ADD THREE ROWS, THEN PREDICT WHAT **CTRL+U** WILL SHOW. Decide before you press it.

EVERY NODE IS A NODE. NOT EVERY NODE IS AN ELEMENT.

```
1 stage.childNodes      // NodeList(5) - text, h3, text, ul, text
2 stage.children        // HTMLCollection(2) - h3, ul
3 stage.childElementCount // 2
4
5 stage.firstChild      // #text - the indentation in your HTML file
6 stage.firstElementChild // <h3> - what you meant
7
8 // the whitespace between tags is a REAL node, and it is why
9 // firstChild surprises people the first time
```

The indentation you write for humans is data the browser keeps.

THE SCRIPT THAT RAN BEFORE THE PAGE EXISTED

THE BUG

```
1 <head>
2   <script>
3     document.querySelector('#list').append(row);
4   </script>
5 </head>
6 <body>
7   <ul id="list"></ul>
8   <!-- TypeError: ... of null -->
9 </body>
```

THE FIX

```
1 <head>
2   <script type="module" src="js/app.js"></script>
3 </head>
4 <body>
5   <ul id="list"></ul>
6 </body>
7
8 <!-- a module is deferred: it runs after the
9      document is parsed. So is defer. -->
```

YOU ALREADY KNOW THE SELECTOR LANGUAGE

the week-3 selectors, handed to JavaScript↶ reloadopen ↷

ONE SELECTOR, FOUR QUESTIONS THE PAGE ANSWERS FOR YOU

selector

run add a card reset

.card p

.card > p

p.meta

#first-card

article .sale

article:first-of-type h4

.card:not(.wide)

.cart

button.buy

#

THE MARKUP BEING SEARCHED

Coffee grinder

Burr, 40 steps, quiet.

In stock · 89

Kettle

Gooseneck, variable temperature.

Backorder · 120

Scale

0.1 g, two-year battery.

In stock · 45

WHAT THE BROWSER ANSWERED

`querySelectorAll` → `NodeList(6)`

`querySelector` → `p.match.first` – the first match in document order, highlighted in yellow

Matched: `p.match.first` · `p.meta.match` · `p.match` · `p.meta.match` · `p.match` · `p.meta.match`

Both of these were taken before you pressed anything: `getElementsByClassName('card')` → `3` · `querySelectorAll('.card')` → `3`. Press "add a card" and watch which of the two numbers changes on its own.

► Type `.card p`, then `.card > p`. **EXPLAIN THE DIFFERENCE FROM THE HIGHLIGHT COUNT ALONE.**

← SPINE · 4

6

SELECTING

CALL	RETURNS	NOTES
<code>querySelector(sel)</code>	Element or <code>null</code>	First match in document order. Any CSS selector.
<code>querySelectorAll(sel)</code>	NodeList (static)	A snapshot. <code>forEach</code> works; <code>map</code> does not — spread it first.
<code>el.querySelector(sel)</code>	Element or <code>null</code>	Scoped to <code>el</code> . <code>:scope > li</code> for direct children.
<code>getElementById(id)</code>	Element or <code>null</code>	The <code>id</code> , not a selector. No <code>#</code> .
<code>getElementsBy...</code>	HTMLCollection (live)	<code>ClassName</code> and <code>TagName</code> . Re-answer every read.

WHY `getElementById` IS STILL HERE

What it is The original lookup, from 1998. One id, one element, no selector parsing.

Key trait Marginally faster, and **it cannot be given a wrong selector** — there is no syntax to get wrong.

Use cases Reading it in code that predates you.
Writing it when the thing genuinely has an id.

Tips Use `querySelector` by default: one function, every case, the language you already know.

THE GOTCHA WORTH KNOWING

`getElementById('list')` takes the **id**.

`querySelector('#list')` takes a **selector**.

Passing `'#list'` to the first one silently finds nothing, forever.

ONE OF THESE KEEPS LOOKING. THE OTHER TOOK A PHOTOGRAPH.

```
1  const live = document.getElementsByClassName('card'); // HTMLCollection
2  const snap = document.querySelectorAll('.card');      // NodeList
3
4  addOneMoreCard();
5
6  live.length    // 4  - it re-answers the question every time you ask
7  snap.length    // 3  - it was true when you asked, and never again
```

TRAVERSING FROM AN ELEMENT

parentElement · children · nextElementSibling ·
closest(sel) · matches(sel)

THE ONE YOU WILL USE TODAY

closest() walks **up** until something matches. Cycle 3
is built on it.

TRAVERSING: DOWN AND ALONG

PROPERTY	GIVES YOU
<code>parentElement</code>	The element above. <code>parentNode</code> can also be the document.
<code>children</code> · <code>childElementCount</code>	Element children only. Live collection.
<code>childNodes</code>	Everything, including whitespace text nodes and comments.
<code>firstElementChild</code> · <code>lastElementChild</code>	What you usually mean by first and last.
<code>firstChild</code> · <code>lastChild</code>	Often the indentation in your file.
<code>nextElementSibling</code> · <code>previousElementSibling</code>	Along one level



TRAVERSING: UP, AND ASKING QUESTIONS

CALL	GIVES YOU
<code>closest(sel)</code>	Up until something matches, itself included. <code>null</code> if nothing does.
<code>matches(sel)</code>	<code>true</code> / <code>false</code> . Does this element match that selector?
<code>contains(other)</code>	Is <code>other</code> inside this one? Useful for outside-click handlers.
<code>isConnected</code>	Is this element in the document at all? The answer to “I built it and cannot see it”.

CREATING AND INSERTING

CALL

`document.createElement(tag)`

`document.createTextNode(s)`

`parent.append(a, b, 'text')`

`parent.prepend(x)`

`el.before(x) · el.after(x)`

`el.replaceWith(x)`

WHAT IT DOES

A new element, **not** in the document. Nothing shows until it is attached.

A text node. [append](#) accepts a plain string instead.

At the end. Several at once, strings allowed.

At the start.

As a sibling, in front of or behind.

Swap this element for another.

append TAKES STRINGS TOO – WHICH IS HOW undefined GETS IN

CALL	WHAT LANDS IN THE LIST	CONSOLE
<code>list.append(undefined)</code>	the text <code>undefined</code>	nothing
<code>list.append(null)</code>	the text <code>null</code>	nothing
<code>list.append(li, 'x', 3)</code>	the <code>li</code> , then two text nodes, <code>x</code> and <code>3</code>	nothing
<code>list.appendChild(undefined)</code>	nothing	<code>TypeError</code> — Failed to execute 'appendChild' on 'Node': parameter 1 is not of type 'Node'.

► The DOM Standard: every argument of `append` that is not a node is **CONVERTED TO A STRING** and wrapped in a text node. A function that forgot its `return` hands it `undefined`, and the word appears on the page. The 1998 `appendChild` takes one node and refuses anything else.

THREE WAYS TO ADD A ROW. ONE OF THEM KILLS YOUR LISTENERS.

same result on screen, three different documents underneath↻ reloadopen ↗

SAME RESULT ON SCREEN · THREE DIFFERENT DOCUMENTS UNDERNEATH

add one row to each listtime 200 insertsreset

```
list.innerHTML += '<li>_</li>'
```

Reads the whole list back as text, appends to that string, and hands the result to the parser.

row 1 — the one with the handler

rows1

```
list.insertAdjacentHTML('beforeend', ...)
```

Parses the new fragment only, and puts it where you said. Existing children are not touched.

row 1 — the one with the handler

rows1

```
list.append(document.createElement(...))
```

Builds one element object and attaches it. No HTML is parsed at all, so no string can turn into markup.

row 1 — the one with the handler

rows1

Press **add one row**. Before each insert a click handler is attached to the first row of every list, and afterwards the page dispatches a real click on it.

► **BEFORE PRESSING: DECIDE WHICH COLUMN WILL BREAK.** Most people expect the third.

THE LOOP THAT DESTROYS WHAT IT JUST BUILT

THE BUG

```
1 for (const item of items) {  
2   list.innerHTML += `<li>${item.title}</li>`;  
3 }  
4  
5 // every pass re-reads the whole list as a string,  
6 // re-parses it, and throws away every element  
7 // that was there – with its listeners.
```

THE FIX

```
1 list.replaceChildren();  
2 for (const item of items) {  
3   list.append(itemRow(item));  
4 }  
5  
6 // or, when it must be a string:  
7 list.insertAdjacentHTML('beforeend', html);
```

The rows look identical. They are different objects, and only one set has your listeners.

REMOVING, AND PUTTING THINGS IN THE RIGHT PLACE

```
1 row.remove();           // the element removes itself
2
3 list.replaceChildren();  // empty the container
4
5 list.prepend(newest);    // first child instead of last
6
7 target.before(banner);   // as a sibling, in front
8 target.after(banner);    // as a sibling, behind
9
10 list.insertAdjacentHTML('afterbegin', html); // 4 positions, one call
```

THE OLD WAY YOU WILL MEET IN OLD CODE

`parent.removeChild(child)` and `parent.appendChild(one)`. They work. They need the parent, and they take one node.

EMPTYING, REMOVING, COPYING

CALL

`parent.replaceChildren(...)`

`el.remove()`

`el.cloneNode(deep)`

The 1998 trio you will read in old code: `appendChild`, `removeChild` and `insertBefore(x, ref)`. One node each, parent required.

WHAT IT DOES

Empty it, and optionally refill it. **This is how `render()` starts.**

Take this element out. No parent needed.

A copy. `deep = true` copies the children. **Listeners are never copied.**

WORKED EXAMPLE

RENDER THE LIST FROM THE ARRAY

- 1 Open the week-8 page. Delete every `<li>` from the markup. The list is empty and the page still loads.
- 2 Write `itemRow(item)` — four `createElement` calls and a `return`.
- 3 Write `render()` — `replaceChildren()`, a loop, `append`.
- 4 Add a fourth item to the array.
Refresh.

► That last step is the opening problem — two copies of the truth — gone. **NOTICE IT WHEN IT HAPPENS.**

THREE PROPERTIES, AND THEY ARE NOT INTERCHANGEABLE

`textContent`

Reads and writes **text**. Never parses. Fast, because it does not have to lay anything out.

`innerHTML`

Hands the string to the **HTML parser**. Necessary sometimes. Never for text a user typed.

`innerText`

Reads what is **rendered** — skips hidden elements, respects CSS. To know that, it must lay the page out first.

► The performance difference is a footnote. **WHAT EACH ONE READS IS THE REASON.**

THE REASON, AND IT IS NOT THE SPEED

one string, three properties, and a counter on **window**

↺ reload

open ↗

ONE STRING · THREE PROPERTIES · ONE OF THEM EXECUTES IT

the "review" a user typed

assign to all three

plain text

markup, not script

a script tag (does nothing!)

the one that actually runs

and it is not about images

box.textContent = s

Great grinder, 5 stars

child elements created

0

shows the characters you typed

yes

box.innerHTML = s

Great grinder, 5 stars

child elements created

0

shows the characters you typed

yes

box.innerText = s

Great grinder, 5 stars

child elements created

0

shows the characters you typed

yes

Nothing ran. window.__ran is 0.

INNERTEXT IS NOT TEXTCONTENT, AND THE DIFFERENCE IS CSS

The kettle is LOUD.

PROPERTY	WHAT IT RETURNED	LENGTH
textContent	The kettle is loud – internal note, do not ship.	48
innerText	The kettle is loud – internal note, do not ship.	48
innerHTML	The kettle is loud – internal note, do not ship.	101

textContent reads the tree. innerText reads the rendered result, so it drops the hidden span, and to know what is hidden it has to lay the page out first. That is the performance footnote — and it is a footnote.

► **PRESS THE <SCRIPT> PRESET FIRST AND NOTICE HOW HARMLESS IT LOOKS.** Then press the next one.

← SPINE · 23

CONTENT: FOUR WAYS IN, AND ONLY TWO YOU SHOULD USE

PROPERTY	READS	WRITES	USE IT?
<code>textContent</code>	All text, hidden included	Text. Never parsed.	Yes — the default.
<code>innerText</code>	Text as rendered : CSS applied	Text, and forces a layout	Only for what the user sees.
<code>innerHTML</code>	The markup inside	Parsed as HTML	Never with text a user typed.
<code>insertAdjacentHTML(pos, s)</code>	—	Parses only the new fragment	Existing children survive.
<code>insertAdjacentText(pos, s)</code>	—	Text, at a position	Safe version of the row above.

`pos` is one of `beforebegin` · `afterbegin` · `beforeend` · `afterend`.

STATE GOES IN CLASSES. VALUES GO IN `style`.

```
1 row.classList.add('selected');
2 row.classList.remove('selected');
3 row.classList.toggle('selected');           // and toggle(name, force)
4 row.classList.contains('selected');         // true / false
5
6 // a computed value has nowhere else to go:
7 bar.style.width = `${percent}%`;
8
9 // this is not a value you computed. it is a decision.
10 badge.style.backgroundColor = 'gold';       // now you have a problem
```

A class is a decision the stylesheet can argue with. An inline style is a fact it cannot.

WHAT AN INLINE STYLE TAKES AWAY

add a class, then write one inline style, then add a class again

reload

open

A CLASS IS A DECISION · AN INLINE STYLE IS A FACT YOU CANNOT ARGUE WITH

A · STYLING THE BADGE

in stock

classList.add('ok')

classList.add('danger')

classList.toggle('danger')

style.backgroundColor = 'gold'

now try a class again

reset

class attribute	(none)
classList.contains("danger")	false
style.backgroundColor (inline only)	(empty string)
getComputedStyle → background	rgb(241, 245, 249)
dataset.sku · dataset.unitPrice	GR-40 · 89

Press the class buttons first. Then press the inline one, then **now try a class again**.

[a relative link, for the third pair](#)

B · THE ATTRIBUTE IS NOT THE PROPERTY

the markup says value="pencil" and no checked

quantity note pencil

☐ gift wrap

READ AS	PROPERTY	GETATTRIBUTE
input.value	"pencil"	"pencil"
checkbox.checked	false	null
a.href	"http://127.0.0.1:56024/weeks/week-09-dom-debug/demos/01-live-tree/index.html"	"../01-live-tree/index.html"
badge.dataset.sku	"GR-40"	"GR-40"

Yellow means the two disagree. The attribute is the **HTML file**; the property is the **element right now**. A form is read through properties — always.

► The class **IS** added — the class attribute proves it — and the colour does not move.

CLASSES, STYLES, ATTRIBUTES, DATA

CALL

`classList.add / remove / toggle / contains`

`classList.replace(a, b)`

`el.style.prop = v`

`getComputedStyle(el).prop`

`getAttribute / setAttribute / removeAttribute`

`hasAttribute(name)`

`el.dataset.thing`

`el.hidden = true`

NOTES

`toggle(name, force)` takes a boolean: one call for both directions.

Swap one for another.

Writes an **inline** style, which beats every stylesheet. For computed values only.

What is actually applied. `el.style` only reads inline.

The HTML attribute — what the file said.

Presence, for boolean attributes like `hidden`, `disabled`.

`data-thing`, both ways. `data-unit-price` is `dataset.unitPrice`.

A property with an attribute behind it. The empty state, in one line.

hidden **LOSES TO YOUR OWN** display

STILL ON SCREEN

```
<style>
  .empty { display: flex; }
</style>
<p class="empty" hidden>Nothing here yet.</p>

<!-- computed display: flex · height 18px -->
```

HIDDEN AGAIN

```
/* one line in your sheet, once */
[hidden] {
  display: none !important;
}

/* computed display: none · height 0 */
```

- The browser hides `[hidden]` with `display: none` from its **OWN** stylesheet. Any `display` in yours wins the cascade — origin before specificity, week 3 — and `empty.hidden = true` changes the attribute and nothing on screen.

THE ATTRIBUTE IS THE FILE. THE PROPERTY IS THE ELEMENT.

```
1 // markup: <input id="q" value="pencil"> ... then a user types "notebook"
2 q.value // "notebook" ← what is on screen
3 q.getAttribute('value') // "pencil" ← what the file said
4
5 gift.checked // true
6 gift.getAttribute('checked') // null - it never changed
7
8 link.href // "http://localhost:5173/about.html" (resolved)
9 link.getAttribute('href') // "about.html" (as written)
10
11 row.dataset.id // "a1" ← data-id="a1", both ways
12 row.dataset.unitPrice // data-unit-price, camelCased
```

A form is read through properties. Always.

LISTENING, AND THE PARENTHESES THAT RUIN IT

THE BUG

```
1 button.addEventListener('click', handleClick());
2
3 // handleClick RUNS, right now, and its
4 // RETURN VALUE – probably undefined – is
5 // registered as the listener. nothing
6 // happens when you click. nothing errors
7 // either.
```

THE FIX

```
1 button.addEventListener('click', handleClick);
2
3 button.addEventListener('click', (event) => {
4   handleClick(event, extraThing);
5 });
```

You are handing over the function itself. Parentheses call it.

THE EVENTS YOU WILL ACTUALLY USE

EVENT	FIRES WHEN	WATCH OUT
<code>click</code>	Pointer or keyboard activation of a button or link	Enter and Space fire it on a real <code><button></code> . On a <code><div></code> they do not.
<code>submit</code>	A form is submitted, by button or by Enter	On the form , never on the button.
<code>input</code>	Every keystroke, paste and undo	What you want for live filtering.
<code>change</code>	Value committed: blur for text, immediately for checkbox and select	Two different timings under one name.
<code>keydown</code>	A key goes down. <code>event.key</code> is <code>'Enter'</code> , <code>'Escape'</code> , <code>'a'</code>	<code>keypress</code> is dead. Do not use it.
<code>focus</code> · <code>blur</code>	An element gains or loses focus	They do not bubble. Use <code>focusin</code> / <code>focusout</code> for delegation.
<code>pointerdown</code> / <code>pointerup</code> cover mouse, touch and pen in one API — prefer them over <code>mousedown</code> for anything draggable.		

KEYBOARD: `key`, `code`, AND WHERE TYPING HAPPENS

PROPERTY	WHAT IT HOLDS	MEASURED
<code>event.key</code>	The character the key produced, in the active layout , or its name	<code>'n'</code> · <code>'N'</code> with Shift · <code>'Enter'</code> · <code>' '</code> for Space · <code>'Escape'</code>
<code>event.code</code>	The physical key, whatever the layout	<code>'KeyN'</code> for both <code>n</code> and <code>N</code> · <code>'Space'</code>
<code>shiftKey</code> · <code>ctrlKey</code> · <code>altKey</code> · <code>metaKey</code>	Which modifiers were held	<code>shiftKey</code> is <code>true</code> for <code>'N'</code>
<code>event.repeat</code>	<code>true</code> while a held key auto-repeats	a shortcut should usually ignore it
<code>event.target</code>	The focused element — where the typing is	guard: <code>matches('input, textarea, select')</code> , plus <code>isContentEditable</code> if the page has one

► **ON A HEBREW LAYOUT** the key labelled N produces `key: 'n'` and still `code: 'KeyN'`. A shortcut matched on `key` follows the layout; one matched on `code` follows the key.

ADDEVENTLISTENER, IN FULL

```
1 target.addEventListener(type, handler, options);
2
3 target.addEventListener('click', onClick, {
4   capture: false,    // true = hear it on the way DOWN
5   once: true,        // remove itself after the first time
6   passive: true,     // promises not to call preventDefault (scroll performance)
7   signal: controller.signal, // remove it by aborting the controller
8 });
9
10 // removing needs the SAME function reference:
11 target.removeEventListener('click', onClick);           // works
12 target.removeEventListener('click', () => {});           // does nothing at all
13
14 // one listener, removed later, without keeping the reference:
15 const c = new AbortController(); /* ... */ c.abort();
```

THE EVENT OBJECT

FIELD	WHAT IT IS
<code>type</code>	'click', 'submit' — useful when one handler serves several.
<code>target</code>	What was hit. Often deeper than you expect.
<code>currentTarget</code>	Who is listening. The element you attached to. Never changes.
<code>eventPhase</code>	1 capturing · 2 at the target · 3 bubbling.
<code>key</code> · <code>code</code>	Keyboard. <code>key</code> is the character, <code>code</code> is the physical key.
<code>preventDefault()</code>	Cancels what the browser would have done.
<code>stopPropagation()</code>	Stops other listeners above from hearing it. Rarely correct.
<code>defaultPrevented</code>	Did someone already call <code>preventDefault</code> ?
<code>composedPath()</code>	The full array of nodes the event travels through. Excellent for debugging.

ONE CLICK, SEVEN LISTENERS, ONE ORDER

capture goes down, bubble comes back up – and the browser chose the order, not this page ↻ reload open ↗

ONE CLICK · SEVEN LISTENERS · ONE ORDER, AND THE BROWSER CHOSE IT

☐ `stopPropagation()` in the middle box, bubble phase clear the log

OUTER · DIV#OUTER

MIDDLE · DIV#MIDDLE

INNER · DIV#INNER

Add to cart

Click the button. Then click the green area around it, then the blue area. The target column changes; the `currentTarget` column is the listener that is speaking.

WHAT FIRED, IN ORDER

#	PHASE	CURRENTTARGET	TARGET
phase 1 = capturing (going down) · 2 = at the target · 3 = bubbling (coming back up)			

► **GUESS HOW MANY HANDLERS RAN, THEN COUNT.** It is seven.

preventDefault AND stopPropagation ARE NOT A PAIR

preventDefault()

Stops what the **browser** would do: submit the form, follow the link, tick the box. Uses it every day.

stopPropagation()

Stops **other code** from hearing the event. Breaks the outside-click handler that closes a menu. Almost never the answer.

THE TELL

Reaching for `stopPropagation` to stop a handler firing twice means **you registered it twice**. Fix that.

They sound similar and they cancel completely different things.

this INSIDE A LISTENER

```
1 button.addEventListener('click', function () {
2   this;           // the button - addEventListener sets it
3   this.disabled = true;
4 });
5
6 button.addEventListener('click', () => {
7   this;           // NOT the button. an arrow has no `this` of its own,
8                   // so it is whatever `this` was where you WROTE it.
9 });
10
11 // the rule that removes the question entirely:
12 // use event.currentTarget, and never think about it again.
```

Week 8 said the call site decides.
`addEventListener` is a call site.



DELEGATION, AS A RECIPE

```
1 container.addEventListener('click', (event) => {  
2   const hit = event.target.closest('[data-action="remove"]');  
3   if (!hit) return; // the click was not for us  
4  
5   const id = hit.closest('li').dataset.id; // identity from the DOM  
6  
7   removeItem(id); // change the data  
8   render(); // redraw from the data  
9 });
```

ATTACH TO

A container **render()** empties but **never replaces** — the `<ul>` itself, or a wrapper.

DO NOT ATTACH TO

`document` by default. It works, and every click in the page now runs your code.

SEVERAL ACTIONS

One listener, `data-action` on each control, and a `switch`. Not one listener per action.

COUNTING THE LISTENERS

WHERE	WHAT IT SHOWS	WORTH KNOWING
Elements, Event Listeners tab	Every listener that would hear an event on the selected element, grouped by type, each with the node it is attached to	Untick Ancestors to see only the element's own. A delegated listener shows up under the <code>ul</code> , not the button
Console: <code>getEventListeners(\$0)</code>	The same, as an object: <code>{ click: [...] }</code>	A DevTools console utility — it does not exist in your page's code
Console: <code>monitorEvents(\$0, 'click')</code>	Logs every click on that element, with the event	<code>unmonitorEvents(\$0)</code> stops it
Sources, Event Listener Breakpoints	Pause on the first line of whatever handler runs for, say, <code>click</code>	The answer to "is my handler even running?"

► Cycle 4, measured: the answer to the naive prompt registered `click` **4, 8, 11** times after load, one add and one removal; the constrained one, **2** — the form and the list — and never again.

FORMS

CALL

`input.value`

`checkbox.checked` · `radio.checked`

`select.value` · `select.selectedIndex`

`form.elements.name` · `form.name`

`new FormData(form)`

`form.reset()`

`form.requestSubmit()`

`input.setAttribute('aria-invalid', 'true')`

`input.focus()`

NOTES

Always a string, including `type="number"`. Convert at the boundary.

Boolean property. The `checked` attribute is only the initial state.

The chosen option's value.

Reach a field by its `name` attribute.

Every named field at once. `Object.fromEntries(fd)` gives a plain object.

Back to the initial values. Does not fire `input`.

Submit as if a button was pressed — validation and the `submit` event included.

What a screen reader announces. Set it **and clear it**.

Where the user types next. The third part of failing well.

WORKED EXAMPLE

THE ADD FORM, FROM NOTHING

- 1 `form.addEventListener('submit', ...)` and `event.preventDefault()`. Press the button: nothing reloads.
- 2 Read both fields. `Number()` the second one, out loud, at the boundary.
- 3 Push into the array, `render()`. A row appears.
- 4 Now submit with an empty name — and watch it push an empty row.
Then fix it.

► **DO STEP 4 FOR REAL.** Validation that arrives after the bug is remembered; validation that arrives first is a rule you will forget.

ELEMENTS: THE DOM, EDITABLE, IN FRONT OF YOU

WHAT IT SHOWS

The **document**, not the file.
Everything JavaScript added is here.

LIVE EDIT

Double-click a class, an attribute, a text node. The page changes.
Nothing is saved — refresh and it is gone.

STYLES & COMPUTED

Which rule won, which lost with a strike-through, and the final number after everything.

► Select an element, switch to Console, type `$0`. **IT IS THE ELEMENT YOU JUST CLICKED.**

THE CONSOLE HAS FIVE MORE THINGS IN IT

each one answers exactly one question

↻ reload

open ↗

SIX CALLS, THE QUESTION EACH ONE ANSWERS, AND THE OUTPUT MIRRORED ON THE PAGE

A card with two data- attributes, for dir to look at.

console.table(items)

console.log(el) vs console.dir(el)

console.group()

console.count()

console.time()

console.trace()

MIRRORED OUTPUT — THE CONSOLE HAS THE SAME THING IN IT

Open DevTools (F12 · Cmd+Option+I), then press a button.

WHICH ONE ANSWERS WHICH QUESTION

QUESTION	CALL
what is in this array of objects?	console.table(list)
what properties does this element have?	console.dir(el)
which of these forty lines belong together?	console.group / groupEnd
how many times did this really run?	console.count(label)
how long did that take?	console.time / timeEnd
who called this function?	console.trace()

Two more that only exist in the console itself, so they cannot be on this page: \$0 is the element you last selected in the Elements panel, and \$\$('li.card') is querySelectorAll with less typing. Select something in Elements, switch to Console, type \$0.

► **CONSOLE.LOG(EL) AND CONSOLE.DIR(EL) ARE THE SAME CALL WITH DIFFERENT ANSWERS.**

SOURCES: STOPPING TIME

BREAKPOINT

Click the line number. Execution stops **before** that line, with every variable in scope readable.

CONDITIONAL

Right-click the line:

`item.id === 'a4'`. Stops on the one iteration you care about, out of forty.

WATCH

An expression re-evaluated at every pause. `guests.map(g => typeof g.seats)` is a whole diagnosis.

► **STEP OVER** runs the next line · **STEP INTO** goes inside the call · **STEP OUT** finishes it · and the **CALL STACK** panel is who is waiting for you.

NETWORK: WHEN IT IS NOT YOUR JAVASCRIPT

STATUS

404 — the path is wrong. **200 but empty** — the path is right and the file is not what you think.

TYPE FILTER

Filter to `JS` and see whether your module was even requested. A typo in `src` is silent otherwise.

THE ONE THAT WASTES AN AFTERNOON

The file loaded — from the **cache**. Tick **Disable cache** while DevTools is open, permanently.

Before debugging what your code did with the data, check that the data arrived.

DEVTOOLS, BY PANEL

PANEL	ANSWERS	WORTH KNOWING
Elements	What is on the page right now?	Live-edit anything. <code>\$0</code> in the console is the selected element.
Styles / Computed	Which rule won?	Struck-through rules lost. Computed is the final number.
Console	What is this value?	<code>table · dir · group · count · time · trace · \$\$()</code>
Sources	What happened, step by step?	Breakpoints, conditional breakpoints, Watch, Call Stack, Event Listener Breakpoints.
Network	Did the file even arrive?	Status, type filter, and Disable cache — leave it ticked.
Application	What is stored?	localStorage lives here. Week 11.

► F12 or Cmd+Option+I · Ctrl/Cmd+Shift+C picks an element · Esc opens the console drawer inside any panel.

THE SIX STEPS, AND WHAT EACH ONE COSTS

STEP	THE QUESTION IT ANSWERS	WHAT GOES WRONG
1 · Reproduce	What exact sequence makes it happen?	“Sometimes it breaks” is not a bug report, even to yourself.
2 · Isolate	Which file, function and line?	Editing before locating. Two changes, no information.
3 · Hypothesise	“X is Y when it should be Z.”	A guess you cannot disprove. It cannot be tested, so it cannot be wrong.
4 · Verify	Is that sentence true?	Skipping straight to the fix. The bug survives and now the code is different.
5 · Fix	What is the cause?	Fixing the symptom — changing the id in the HTML instead of the typo in the selector.
6 · Confirm	Is it gone, and is anything beside it broken?	The step that disappears when time runs out.

WHERE TO LOOK, IN ORDER

#	IF...	LOOK AT
1	There is a red line in the console	It, first. Click the file:line link — it opens Sources on the line.
2	Nothing happens, and nothing errors	Is the handler running at all? One breakpoint on its first line.
3	The handler runs and does the wrong thing	What are the values? Watch, not <code>console.log</code> .
4	What is on screen disagrees with the data	<code>console.table(items)</code> , and count the rows.
5	It works sometimes	What is different about the times it does? Usually <code>event.target</code> .
6	It worked and then stopped	Did something re-render? Every element is new after that.
7	Your change had no effect at all	Network: did the file load? Is Disable cache ticked?

► **THIS IS THE SLIDE TO READ FIRST WHEN SOMETHING IS BROKEN**, and the five faults in the Debug Challenge are rows 1, 2, 3, 4 and 5.